Mobile Application Penetration Testing

Mobile Vulnerabilities (OWASP Mobile Top 10)

Mobile application vulnerabilities refer to security weaknesses that can be exploited to access sensitive data or compromise application functionality. The **OWASP Mobile Top 10** highlights the most critical risks in mobile applications.

Key Vulnerabilities

- M1: Improper Platform Usage
  This vulnerability occurs when mobile applications incorrectly use platform features such as permissions, intents, or security controls.
  - Apps may request excessive permissions
  - Misuse of platform APIs (camera, storage, sensors)
  - Insecure handling of OS-level security mechanisms

- M2: Insecure Data Storage
  This occurs when sensitive data is stored on the device without proper protection.
  - Data stored in plain text
  - Hardcoded credentials in the application
  - Sensitive information stored in logs or cache

Example: Extracting Sensitive Data from Android APK
Android applications are packaged as APK files, which can be reverse engineered to reveal internal code and data.

Process

- Obtain the APK file
- Decompile the APK using analysis tools
- Examine files for:
  - API keys
  - User credentials
  - Database information

Impact

- Exposure of sensitive data
- Unauthorized access to backend systems
- Increased risk of further exploitation

Prevention

- Encrypt sensitive data before storage
- Avoid hardcoding secrets in the app
- Implement code obfuscation
- Follow secure mobile development practices

Testing Techniques in Mobile Application Pentesting

Testing techniques in mobile penetration testing involve analysing applications to identify vulnerabilities using both static and dynamic approaches. These methods help uncover security flaws in code, data handling, and runtime behaviour.

Static Analysis

Static analysis is the process of examining the application without executing it. It focuses on reviewing the source code or decompiled APK to identify security issues.

Tool Example

MobSF

- Decompiles APK/IPA files
- Detects hardcoded credentials and insecure configurations
- Provides detailed vulnerability reports

Purpose

- Identify coding flaws
- Detect insecure data storage
- Find embedded secrets like API keys

Dynamic Testing

Dynamic testing involves analysing the application while it is running on a device or emulator. It helps observe real-time behaviour and interactions.

Tool Example

Frida

- Intercepts function calls
- Modifies app behaviour at runtime
- Bypasses security mechanisms (e.g., root detection, SSL pinning)

Purpose

- Monitor runtime data flow
- Test authentication and session handling
- Identify vulnerabilities not visible in static analysis

Static vs Dynamic Analysis

Feature	Static Analysis	Dynamic Testing
Execution Needed	No	Yes
Focus	Code & structure	Runtime behaviour
Tools	MobSF	Frida
Use Case	Early vulnerability detection	Real-time attack simulation

Table-1 Static vs Dynamic Analysis

Secure Mobile Design

Secure mobile design focuses on implementing security measures during application development to protect sensitive data and prevent exploitation. It ensures that mobile applications are resilient against common attacks.

Key Security Mechanisms

- Secure Storage

  Secure storage ensures that sensitive data is protected when stored on the device.
  - Encrypt data using strong encryption algorithms
  - Use secure storage mechanisms (e.g., Keychain, Keystore)
  - Avoid storing sensitive data in plain text

- Code Obfuscation

  Code obfuscation makes application code difficult to understand and reverse engineer.
  - Renames variables and functions
  - Hides application logic

- Reduces readability of decompiled code

❖ Runtime Checks
Runtime checks monitor the application during execution to detect suspicious activities.
- Root/jailbreak detection
- Debugging detection
- Emulator detection

Importance
- Protects user data and privacy
- Reduces risk of reverse engineering
- Enhances overall application security
- Prevents unauthorized access and tampering

Mobile Application Penetration Testing

The primary objective of mobile application penetration testing is to develop strong skills in both identifying vulnerabilities and securing mobile applications against potential attacks.

Key Objectives

❖ Identify Vulnerabilities
- Detect security flaws based on the **OWASP Mobile Top 10
- Analyse issues like insecure storage, improper platform usage, and weak authentication

❖ Perform Security Testing
- Conduct static analysis using tools like MobSF
- Execute dynamic testing using Frida
- Evaluate application behavior under real-world attack scenarios

❖ **Exploit Weaknesses**
- Understand how vulnerabilities can be exploited
- Simulate attacks to assess impact and severity

❖ Implement Secure Practices
- Apply secure coding techniques
- Use encryption, obfuscation, and runtime protections

- Strengthen overall application security

Study OWASP Mobile Security Testing Guide

The **OWASP Mobile Security Testing Guide** is a comprehensive resource that provides standardized methodologies for testing the security of mobile applications. It helps security professionals and developers understand how to identify and fix vulnerabilities in Android and iOS apps.

Key Features

Standardized Testing Methodology

- Provides step-by-step procedures for mobile security testing
- Covers both static and dynamic analysis techniques
- Aligns with industry best practices

Comprehensive Coverage

- Authentication and authorization testing
- Data storage and privacy checks
- Network communication security
- Platform-specific vulnerabilities

Practical Testing Approach

- Includes real-world testing scenarios
- Explains how to use tools and techniques effectively
- Helps simulate real attacker behavior

Importance

- Widely accepted industry standard for mobile security testing
- Helps ensure thorough vulnerability assessment
- Improves both offensive and defensive security skills

Review SANS Mobile Pentest Case Studies

The SANS Institute mobile pentest case studies provide real-world insights into how vulnerabilities in mobile applications are identified, exploited, and mitigated. These case studies highlight common issues such as insecure data storage, weak authentication, and improper platform usage, helping learners understand how attackers target mobile apps. They also demonstrate practical attack techniques like reverse engineering APK files and bypassing security controls, making the learning process more hands-on and realistic.

Additionally, these case studies present structured testing methodologies aligned with standards from organizations like **OWASP**, ensuring a systematic approach to mobile security testing. They also emphasize effective mitigation strategies, including secure coding practices and proper data protection methods. Overall, reviewing these case studies helps build strong practical skills in both identifying vulnerabilities and implementing security measures in mobile applications.

Conclusion

Mobile Application Penetration Testing plays a critical role in identifying and mitigating security vulnerabilities that can compromise sensitive data and application integrity. Throughout this project, key risks outlined in the **OWASP Mobile Top 10** were explored, including improper platform usage and insecure data storage. By applying both static analysis using tools like MobSF and dynamic testing with Frida, it was possible to effectively detect, analyse, and simulate real-world attack scenarios.

The project also emphasized the importance of secure mobile design practices such as encryption, code obfuscation, and runtime protection mechanisms. Additionally, leveraging frameworks like the OWASP Mobile Security Testing Guide and reviewing real-world case studies helped build a structured and practical approach to mobile security.

Overall, this project enhanced the understanding of both offensive and defensive security techniques, highlighting the importance of proactive testing and secure development practices in building resilient mobile applications.